

Documento de Especificación de Requisitos del
Sistema MR. Car

Contenido

INTRODUCCIÓN.....	4
1. DESCRIPCIÓN GENERAL.....	5
2. FUNCIONES DEL PRODUCTO.....	6
2.1 Gestión de usuarios y acceso.....	6
2.2 Gestión de talleres.....	6
2.3 Gestión de clientes.....	6
2.4 Gestión de vehículos.....	6
2.5 Gestión de reparaciones.....	6
2.6 Catálogo inteligente.....	7
2.7 Diagramas y pinouts.....	7
2.8 Ubicación de componentes.....	7
2.9. Diagnóstico DTC.....	7
2.10 Personalización.....	7
3. RESTRICCIONES.....	7
3.1 Seguridad.....	7
3.2 Arquitectura.....	7
3.3 Tecnológicas.....	8
3.4 Multimedia.....	8
3.5 Datos.....	8
4. SUPOSICIONES.....	8
5. DEPENDENCIAS.....	8
5.1 Tecnológicas.....	8
5.2 Infraestructura.....	8
5.3.Externas.....	9
5.4.Seguridad.....	9
6. REQUISITOS FUNCIONALES.....	9
6.1. GESTIÓN BASE (USUARIOS, ACCESO Y MULTITENENCIA).....	9
RF01 – Registro de Usuario.....	9
RF03 – Gestión de Roles.....	10
RF04 – Asociación Usuario–Taller.....	11
RF05 – Control de Acceso.....	11
6.2 GESTIÓN DE TALLER.....	12
RF06 – Creación de Taller.....	12
RF07 – Gestión de Usuarios del Taller.....	13
6.3 GESTIÓN DE CLIENTES.....	13
RF08 – Registro de Cliente.....	13

RF09 – Consulta de Clientes.....	14
6.4. GESTIÓN DE VEHÍCULOS.....	15
RF10 – Registro de Vehículo.....	15
RF11 – Consulta de Vehículos.....	15
6.5 GESTIÓN DE REPARACIONES.....	16
RF12 – Creación de Reparación.....	16
RF13 – Actualización de Reparación.....	16
RF14 – Historial de Reparaciones.....	17
6.6. CATÁLOGO INTELIGENTE.....	17
RF15 – Búsqueda por Vehículo (Filtro Cascada).....	18
RF16 – Ficha Técnica ECU.....	18
6.7. DIAGRAMAS Y PINOUTS.....	19
RF17 – Visualización de Pinouts.....	19
RF18 – Visualización de Diagramas.....	19
RF19 – Visualización de Caja de Fusibles.....	20
6.8. UBICACIÓN DE COMPONENTES.....	20
RF20 – Ubicación de Componentes.....	20
6.9. DTC DIAGNÓSTICO.....	20
RF21 – Búsqueda de Código DTC.....	20
6.10. PERSONALIZACIÓN.....	21
RF22 – Gestión de Favoritos.....	21
7. REQUISITOS NO FUNCIONALES.....	21
7.1 RNF01 – Seguridad de Autenticación.....	21
Descripción:	
El sistema debe garantizar que solo usuarios autenticados accedan a los recursos mediante mecanismos seguros.....	21
7.2 RNF02 – Control de Acceso (RBAC).....	22
7.3 RNF03 – Aislamiento Multi-Tenant.....	23
7.4 RNF04 – Rendimiento de Búsqueda.....	23
7.5 RNF05 – Rendimiento General del Sistema.....	24
7.6 RNF06 – Gestión de Contenido Multimedia.....	25
7.7 RNF07 – Escalabilidad Horizontal.....	25
7.8 RNF08 – Consistencia de Datos.....	26
7.9 RNF09 – Disponibilidad del Sistema.....	26
7.10 RNF10 – Integridad de Datos.....	27
7.11 RNF11 – Auditoría y Trazabilidad.....	27
7.12 RNF12 – Protección de Datos Sensibles.....	28
7.14 RNF14 – Mantenibilidad.....	29
7.15 RNF15 – Despliegue Continuo.....	29
8. Modelo de Base de datos.....	30

9. Arquitectura microservicios.....	30
10. Servicio Autenticación.....	30
10.1 Implementación de la autenticación por tokens.....	31
11. Implementación por capas.....	32
12. Swagger.....	40
13. Testing.....	41
Conclusión.....	42

INTRODUCCIÓN

El presente proyecto corresponde al diseño del sistema MR.Car, una plataforma web integral orientada al diagnóstico automotriz y la gestión operativa de talleres mecánicos. La solución se concibe como un sistema tipo SaaS (Software as a Service), capaz de centralizar información técnica especializada junto con herramientas de administración, con el objetivo de optimizar los procesos de diagnóstico, reparación y gestión de clientes en talleres mecánicos.

En la actualidad, los técnicos automotrices enfrentan dificultades asociadas a la dispersión de información técnica, tiempos elevados de búsqueda y falta de herramientas integradas que permitan gestionar tanto el conocimiento como la operación diaria del taller. MR.Car surge como una respuesta a esta problemática, proporcionando un entorno unificado que integra funcionalidades como catálogo inteligente de vehículos, visualización de diagramas eléctricos, consulta de códigos de falla (DTC), ubicación de componentes y gestión de reparaciones.

Desde el punto de vista tecnológico, el sistema ha sido concebido bajo una arquitectura de microservicios con enfoque multi-tenant, permitiendo la coexistencia de múltiples talleres en una misma plataforma con aislamiento seguro de datos. Asimismo, se incorporan mecanismos robustos de autenticación, control de acceso basado en roles y escalabilidad horizontal, asegurando tanto la seguridad como el rendimiento del sistema.

Este documento presenta la definición de requisitos funcionales y no funcionales, el modelo de datos mediante un diagrama entidad-relación, la identificación de riesgos y la propuesta de arquitectura lógica del sistema, constituyendo una base sólida para el desarrollo e implementación de MR.Car.

1. DESCRIPCIÓN GENERAL

MR.Car es una plataforma web integral orientada al diagnóstico automotriz y a la gestión operativa de talleres mecánicos. El sistema está diseñado como una solución SaaS (Software as a Service) multi-tenant, permitiendo que múltiples talleres utilicen la plataforma de manera independiente, con aislamiento seguro de sus datos.

El sistema busca centralizar en un solo entorno:

- Información técnica automotriz (ECU, diagramas, pinouts, DTC)
- Herramientas de gestión (clientes, vehículos, reparaciones)
- Funcionalidades de búsqueda avanzada

MR.Car está dirigido a:

- Mecánicos
- Técnicos automotrices

- Administradores de talleres

Su propósito principal es reducir los tiempos de diagnóstico, mejorar la precisión técnica y optimizar la gestión interna del taller.

2. FUNCIONES DEL PRODUCTO

El sistema MR.Car incluye las siguientes funcionalidades principales:

2.1 Gestión de usuarios y acceso

- Registro y autenticación de usuarios
- Gestión de roles (ADMIN, MECÁNICO)
- Asociación de usuarios a talleres
- Control de acceso basado en roles (RBAC)

2.2 Gestión de talleres

- Creación de talleres
- Administración de usuarios por taller

2.3 Gestión de clientes

- Registro de clientes
- Consulta y filtrado de clientes

2.4 Gestión de vehículos

- Registro de vehículos asociados a clientes
- Consulta de historial por vehículo

2.5 Gestión de reparaciones

- Creación de reparaciones
- Registro de notas técnicas
- Historial de intervenciones

2.6 Catálogo inteligente

- Búsqueda por filtro cascada (marca, modelo, año, motor)
- Acceso a información técnica del vehículo

2.7 Diagramas y pinouts

- Visualización de diagramas eléctricos
- Consulta de pinouts de ECU
- Información de fusibles y relés

2.8 Ubicación de componentes

- Visualización de ubicación de ECU y sensores
- Representaciones gráficas (2D/3D)

2.9. Diagnóstico DTC

- Búsqueda de códigos OBD2
- Información de fallas, causas y síntomas

2.10 Personalización

- Gestión de favoritos
- Historial de uso

3. RESTRICCIONES

El sistema MR.Car debe operar bajo las siguientes restricciones:

3.1 Seguridad

- Uso obligatorio de HTTPS
- Implementación de autenticación mediante JWT
- Encriptación de contraseñas

3.2 Arquitectura

- Arquitectura basada en microservicios
- Separación de dominios (gestión vs técnico)
- Uso de base de datos relacional + motor de búsqueda

3.3 Tecnológicas

- Compatible con navegadores modernos (Chrome, Edge, Firefox)
- Dependencia de conexión a internet

3.4 Multimedia

- Limitación en tamaño de imágenes para optimizar rendimiento
- Uso de CDN para contenido pesado

3.5 Datos

- Integridad referencial obligatoria (PK/FK)
- Aislamiento por taller_id

4. SUPOSICIONES

Para el correcto funcionamiento del sistema, se asumen las siguientes condiciones:

- Los usuarios cuentan con conocimientos básicos en el uso de sistemas web
- Los talleres disponen de conexión a internet estable
- Los datos técnicos automotrices son proporcionados por fuentes confiables
- Los usuarios ingresan información correcta y verificada
- El volumen de usuarios crecerá progresivamente (escalabilidad requerida)
- Las ECUs y diagramas tienen formatos estandarizados

5. DEPENDENCIAS

El sistema depende de los siguientes elementos externos:

5.1 Tecnológicas

- Base de datos (PostgreSQL o similar)
- Motor de búsqueda (ElasticSearch)
- Sistema de mensajería (Kafka o similar)
- Servicios de almacenamiento (S3 o equivalente)

5.2 Infraestructura

- Servicios en la nube (AWS, Azure, GCP)
- CDN para distribución de contenido

5.3.Externas

- Fuentes de datos automotrices (ECU, DTC, diagramas)
- Posibles integraciones futuras con dispositivos OBD2

5.4.Seguridad

- Certificados SSL
- Servicios de autenticación

6. REQUISITOS FUNCIONALES

6.1. GESTIÓN BASE (USUARIOS, ACCESO Y MULTITENENCIA)

RF01 – Registro de Usuario

Descripción:

El sistema debe permitir el registro de nuevos usuarios.

Entradas:

- nombre
- email
- contraseña

Proceso:

- Validar formato email
- Verificar unicidad
- Encriptar contraseña

Salidas:

- Usuario registrado

Reglas de negocio:

- Email único global
- Contraseña mínimo 8 caracteres

Criterio de aceptación:

- Usuario creado correctamente y persistido

RF02 – Autenticación de Usuario (Login)

Descripción:

Permitir a usuarios autenticarse en el sistema.

Entradas:

- email
- contraseña

Proceso:

- Validar credenciales
- Generar JWT

Salidas:

- Token JWT
- Datos del usuario

Reglas de negocio:

- Bloqueo tras 5 intentos fallidos

Criterio de aceptación:

- Token válido generado en login exitoso

RF03 – Gestión de Roles

Descripción:

Asignar roles a los usuarios dentro de un taller.

Entradas:

- usuario_id
- taller_id
- rol

Proceso:

- Validar permisos de asignación

Salidas:

- Rol asignado

Reglas de negocio:

- Solo ADMIN puede asignar roles

Criterio de aceptación:

- Rol correctamente aplicado

RF04 – Asociación Usuario–Taller**Descripción:**

Relacionar usuarios con uno o más talleres.

Entradas:

- usuario_id
- taller_id

Proceso:

- Validar existencia de ambos

Salidas:

- Relación creada

Reglas de negocio:

- Un usuario puede pertenecer a múltiples talleres

Criterio de aceptación:

- Relación persistida correctamente

RF05 – Control de Acceso**Descripción:**

Validar acceso a recursos según rol y taller.

Entradas:

- token
- recurso solicitado

Proceso:

- Validar JWT
- Validar rol
- Validar tenant

Salidas:

- Acceso permitido o denegado

Reglas de negocio:

- Aislamiento total entre talleres

Criterio de aceptación:

- No existe acceso cruzado

6.2 GESTIÓN DE TALLER

RF06 – Creación de Taller

Descripción:

El sistema debe permitir la creación de talleres que funcionarán como unidades organizativas independientes dentro del sistema.

Entradas:

- nombre
- rut_empresa
- dirección

Proceso:

- Validar datos

Salidas:

- Taller creado

Reglas:

- Un usuario creador se asigna como ADMIN

Criterio:

- Taller persistido correctamente

RF07 – Gestión de Usuarios del Taller**Descripción:**

El sistema debe permitir administrar los usuarios asociados a un taller, incluyendo altas, bajas y modificaciones.

Entradas:

- lista de usuarios

Proceso:

- Alta/baja/asignación

Salidas:

- Usuarios actualizados

Reglas:

- Solo ADMIN puede modificar

Criterio:

- Cambios reflejados en sistema

6.3 GESTIÓN DE CLIENTES**RF08 – Registro de Cliente****Descripción:**

El sistema debe permitir registrar clientes asociados a un taller, quienes son los propietarios de los vehículos.

Entradas:

- nombre
- teléfono

- email
- taller_id

Proceso:

- Validación duplicidad

Salidas:

- Cliente registrado

Reglas:

- Cliente pertenece a un solo taller

Criterio:

- Cliente visible solo en su taller

RF09 – Consulta de Clientes

Descripción:

El sistema debe permitir consultar y filtrar clientes registrados dentro de un taller.

Entradas:

- filtros (nombre, teléfono)

Proceso:

- Búsqueda filtrada

Salidas:

- Lista de clientes

Reglas:

- Solo datos del taller

Criterio:

- Resultados correctos

6.4. GESTIÓN DE VEHÍCULOS

RF10 – Registro de Vehículo

Descripción:

El sistema debe permitir registrar vehículos asociados a un cliente dentro de un taller.

Entradas:

- cliente_id
- patente
- marca
- modelo
- año

Proceso:

- Validación unicidad por taller

Salidas:

- Vehículo creado

Reglas:

- Patente única dentro del taller

Criterio:

- Vehículo persistido

RF11 – Consulta de Vehículos

Descripción:

El sistema debe permitir consultar vehículos registrados mediante filtros como cliente o patente.

Entradas:

- filtros

Proceso:

- Búsqueda

Salidas:

- Lista de vehículos

Criterio:

- Resultados correctos

6.5 GESTIÓN DE REPARACIONES

RF12 – Creación de Reparación

Descripción:

El sistema debe permitir registrar una nueva reparación asociada a un vehículo y a un técnico del taller.

Entradas:

- vehiculo_id
- tecnico_id
- notas

Proceso:

- Validar relaciones

Salidas:

- Reparación creada

Reglas:

- Técnico debe pertenecer al taller

Criterio:

- Registro exitoso

RF13 – Actualización de Reparación

Descripción:

El sistema debe permitir actualizar el estado y la información de una reparación existente.

Entradas:

- reparación_id
- estado
- notas

Proceso:

- Actualización

Salidas:

- Reparación actualizada

Criterio:

- Cambios persistidos

RF14 – Historial de Reparaciones**Descripción:**

El sistema debe permitir visualizar el historial completo de reparaciones de un vehículo.

Entradas:

- vehiculo_id

Proceso:

- Consulta

Salidas:

- Historial

Criterio:

- Datos completos

6.6. CATÁLOGO INTELIGENTE

RF15 – Búsqueda por Vehículo (Filtro Cascada)

Descripción:

El sistema debe permitir buscar información técnica mediante filtros jerárquicos (marca, modelo, año, motor).

Entradas:

- marca
- modelo
- año
- motor

Proceso:

- Filtrado jerárquico

Salidas:

- Información técnica

Criterio:

- Resultados coherentes

RF16 – Ficha Técnica ECU

Descripción:

El sistema debe mostrar información detallada de una ECU, incluyendo imágenes y especificaciones.

Entradas:

- ecu_id

Proceso:

- Consulta técnica

Salidas:

- imágenes + datos

Criterio:

- Información completa

6.7. DIAGRAMAS Y PINOUTS

RF17 – Visualización de Pinouts

Descripción:

El sistema debe permitir visualizar la distribución de pines de una ECU y sus funciones eléctricas.

Entradas:

- ecu_id

Proceso:

- Consulta pines

Salidas:

- Lista detallada

Criterio:

- Datos correctos

RF18 – Visualización de Diagramas

Descripción:

El sistema debe permitir visualizar diagramas eléctricos asociados a un vehículo.

Entradas:

- vehiculo_id

Salidas:

- Diagramas interactivos

Criterio:

- Visualización correcta

RF19 – Visualización de Caja de Fusibles

Descripción:

El sistema debe mostrar la ubicación, amperaje y función de los fusibles del vehículo.

Entradas:

- vehiculo_id

Salidas:

- Fusibles + amperaje

6.8. UBICACIÓN DE COMPONENTES

RF20 – Ubicación de Componentes

Descripción:

El sistema debe mostrar la ubicación física de componentes clave del vehículo mediante imágenes o esquemas.

Entradas:

- vehiculo_id

Salidas:

- Imagen / esquema

6.9. DTC DIAGNÓSTICO

RF21 – Búsqueda de Código DTC

Descripción:

El sistema debe permitir ingresar un código OBD2 y obtener información diagnóstica completa.

Entradas:

- código

Salidas:

- diagnóstico completo

6.10. PERSONALIZACIÓN

RF22 – Gestión de Favoritos

Descripción:

El sistema debe permitir a los usuarios guardar y gestionar recursos técnicos como favoritos.

Entradas:

- usuario_id
- recurso_id

Proceso:

- Guardar/eliminar

Salidas:

- Lista personalizada

7. REQUISITOS NO FUNCIONALES

7.1 RNF01 – Seguridad de Autenticación

Descripción:

El sistema debe garantizar que solo usuarios autenticados accedan a los recursos mediante mecanismos seguros.

Mecanismo:

- JWT + Refresh Token
- Hash de contraseñas (bcrypt)

Entradas:

- email
- contraseña

Salidas:

- token JWT válido
- refresh token

Criterio de aceptación:

- 100% de endpoints protegidos requieren token válido
- Tiempo de expiración configurable

Riesgos:

- Robo de tokens
- Ataques de fuerza bruta

Mitigación:

- Rate limiting
- Expiración corta + refresh seguro

7.2 RNF02 – Control de Acceso (RBAC)

Descripción:

El sistema debe controlar el acceso según roles y pertenencia a taller.

Roles:

- ADMIN
- MECÁNICO

Entradas:

- token JWT
- recurso solicitado

Salidas:

- acceso permitido / denegado

Criterio de aceptación:

- 0 accesos cruzados entre talleres
- Validación en backend obligatoria

Riesgos:

- Escalada de privilegios

Mitigación:

- Middleware de autorización
- Validación por taller_id

7.3 RNF03 – Aislamiento Multi-Tenant**Descripción:**

Los datos deben estar completamente aislados entre talleres.

Regla clave:

- Toda entidad debe estar ligada directa o indirectamente a taller_id

Entradas:

- ID usuario
- ID recurso

Salidas:

- datos filtrados por taller

Criterio de aceptación:

- 0 fugas de datos entre tenants

Riesgos:

- Filtración de información sensible

Mitigación:

- Filtros obligatorios en queries
- Auditoría de acceso

7.4 RNF04 – Rendimiento de Búsqueda**Descripción:**

El sistema debe responder rápidamente a consultas del catálogo.

Métrica:

- Tiempo de respuesta < 2 segundos

Entradas:

- parámetros de búsqueda

Salidas:

- resultados indexados

Criterio de aceptación:

- 95% de consultas bajo 2s

Riesgos:

- Latencia por joins complejos

Mitigación:

- Uso de Elasticsearch
- Indexación avanzada

7.5 RNF05 – Rendimiento General del Sistema

Descripción:

El sistema debe responder de manera eficiente bajo carga.

Métrica:

- Tiempo promedio API < 500 ms

Criterio de aceptación:

- Soportar al menos 500 usuarios concurrentes

Riesgos:

- Saturación del backend

Mitigación:

- Cache Redis
- Balanceo de carga

7.6 RNF06 – Gestión de Contenido Multimedia

Descripción:

El sistema debe manejar imágenes técnicas en alta resolución eficientemente.

Entradas:

- imágenes ECU / diagramas

Salidas:

- URLs optimizadas

Criterio de aceptación:

- Carga inicial < 3 segundos

Riesgos:

- Alto consumo de ancho de banda

Mitigación:

- CDN
- Compresión
- Lazy loading

7.7 RNF07 – Escalabilidad Horizontal

Descripción:

El sistema debe poder escalar según demanda.

Criterio de aceptación:

- Incremento de instancias sin downtime

Riesgos:

- Arquitectura monolítica

Mitigación:

- Microservicios
- Contenedores (Docker/Kubernetes)

7.8 RNF08 – Consistencia de Datos

Descripción:

El sistema debe manejar consistencia en entorno distribuido.

Tipo:

- Consistencia eventual

Entradas:

- eventos de actualización

Salidas:

- sincronización entre servicios

Criterio de aceptación:

- Tiempo de sincronización < 5 segundos

Riesgos:

- Datos desincronizados

Mitigación:

- Event-driven architecture
- Kafka

7.9 RNF09 – Disponibilidad del Sistema

Descripción:

El sistema debe estar disponible para los usuarios la mayor parte del tiempo.

Métrica:

- SLA 99.5%

Criterio de aceptación:

- Máx. 3.6 horas de downtime mensual

Riesgos:

- Caídas de servicios críticos

Mitigación:

- Failover
- Monitoreo

7.10 RNF10 – Integridad de Datos**Descripción:**

Los datos deben mantenerse correctos y sin corrupción.

Criterio de aceptación:

- Uso de constraints:
 - PK
 - FK
 - UNIQUE

Riesgos:

- Datos inconsistentes

Mitigación:

- Validación en backend
- Transacciones ACID

7.11 RNF11 – Auditoría y Trazabilidad**Descripción:**

El sistema debe registrar acciones relevantes de usuarios.

Entradas:

- acciones CRUD

Salidas:

- logs auditables

Criterio de aceptación:

- 100% de acciones críticas registradas

Riesgos:

- Falta de trazabilidad

Mitigación:

- Logging centralizado

7.12 RNF12 – Protección de Datos Sensibles**Descripción:**

Los datos sensibles deben estar protegidos.

Criterio de aceptación:

- Encriptación en tránsito (HTTPS)
- Encriptación en reposo (AES)

Riesgos:

- Fugas de datos

Mitigación:

- Certificados SSL
- Gestión segura de claves

7.13 RNF13 – Compatibilidad y Accesibilidad**Descripción:**

El sistema debe ser accesible desde múltiples dispositivos.

Criterio de aceptación:

- Compatible con:
 - Chrome
 - Edge
 - Firefox

Riesgos:

- Mala experiencia de usuario

Mitigación:

- Diseño responsive

7.14 RNF14 – Mantenibilidad

Descripción:

El sistema debe ser fácil de mantener y evolucionar.

Criterio de aceptación:

- Código modular
- Cobertura de tests > 70%

Riesgos:

- Deuda técnica

Mitigación:

- Clean Architecture
- CI/CD

7.15 RNF15 – Despliegue Continuo

Descripción:

El sistema debe permitir despliegues frecuentes sin afectar operación.

Criterio de aceptación:

- Deploy sin downtime

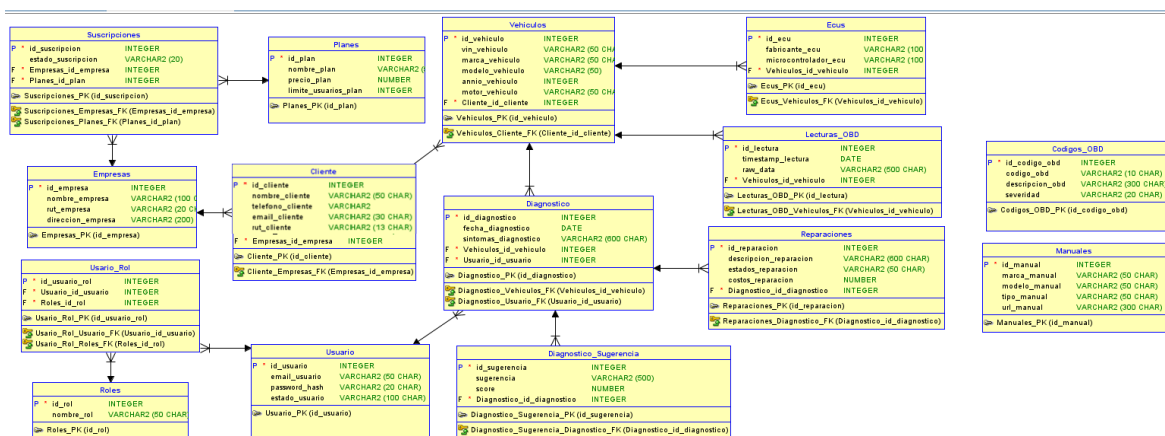
Riesgos:

- Fallos en producción

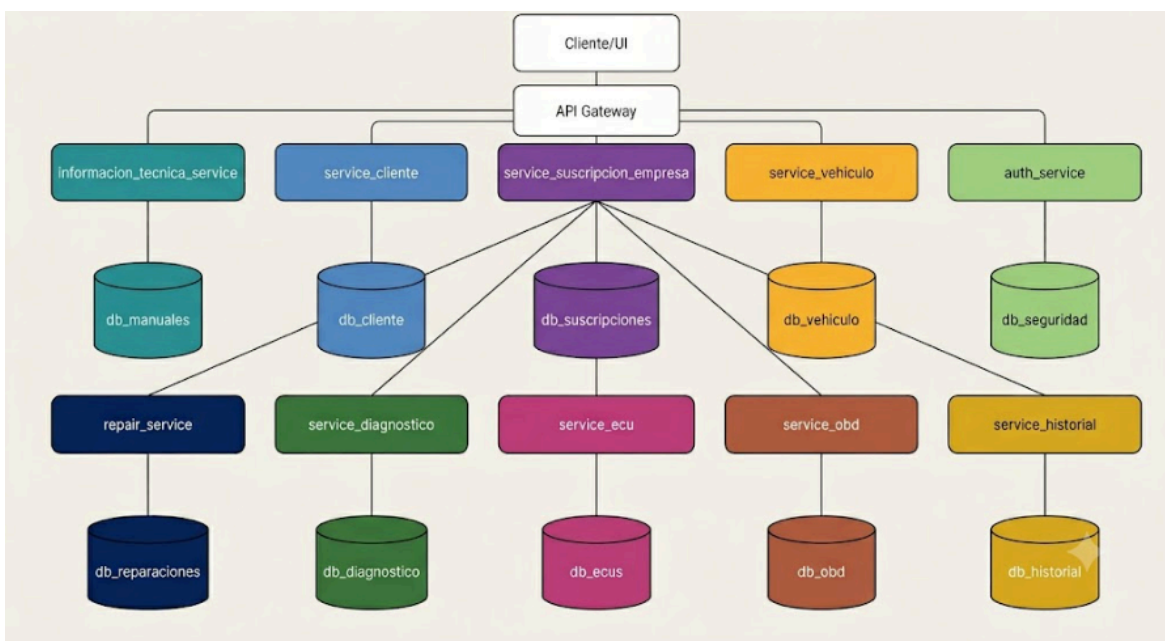
Mitigación:

- Blue/Green deployment
- Canary releases

8. Modelo de Base de datos



9. Arquitectura microservicios



10. Servicio Autenticación

Un servicio de autenticación es un microservicio especializado cuya función es verificar la identidad de los usuarios antes de permitirles acceder a los recursos del sistema.

En una arquitectura de microservicios, este centraliza la autenticación, evitando que cada microservicio tenga que validar usuarios de forma independiente. Su principal responsabilidad es comprobar las credenciales (usuario y contraseña) sean correctas y, una vez verificadas, emitir un token de acceso.

En nuestro proyecto MR.Car, el servicio de autenticación permite que solo usuarios autenticados (clientes, mecánicos o administradores) puedan acceder a los distintos microservicios, como Diagnóstico, ODB, ECU o Historial.

Funciones Principales:

- Validar credenciales de acceso.
- Generar tokens de autenticación.
- Verificar la validez de los tokens.
- Controlar el acceso a los recursos protegidos.
- Centralizar la seguridad de la aplicación.

10.1 Implementación de la autenticación por tokens

La autenticación por token sigue una secuencia de pasos:

a.- Inicio de sesión: El usuario envía sus credenciales mediante una petición POST al servicio de autenticación.

b.-Validación: El microservicio compara las credenciales con las almacenadas en la base de datos. Si son válidas, genera un JWT (JSON Web Token).

c.-Generación del token: El token contiene información como:

- Id del usuario
- Nombre
- Rol
- Fecha de emisión
- Fecha expiración

d.- Respuesta: El servidor devuelve el token al cliente.

e.- Acceso a otros microservicios: En cada petición el cliente envía el token en la cabecera HTTP.

f.- Validación del token: Antes de ejecutar la petición, el sistema verifica:

- Que el token exista;
- Que el token no haya expirado;
- Que la firma sea válida.

Si todo es correcto, el usuario puede acceder al recurso solicitado.

11. Implementación por capas

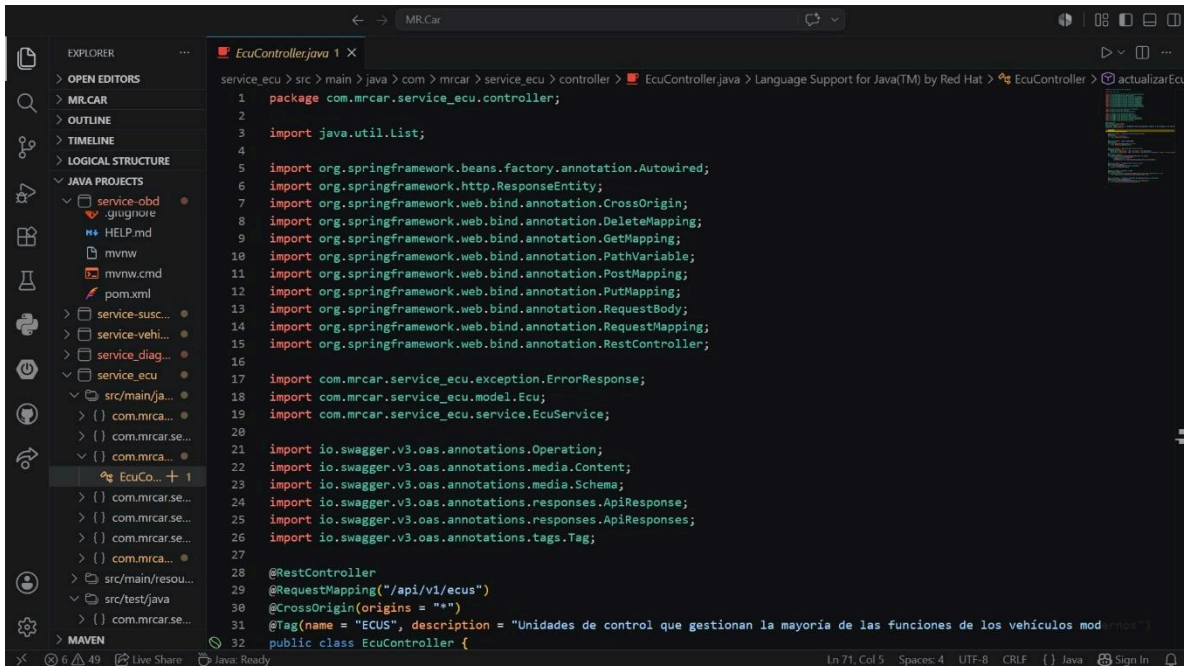
La implementación por capas consiste en dividir la aplicación en diferentes niveles, donde cada uno tiene una responsabilidad específica. Esto facilita el mantenimiento, la reutilización del código y la escalabilidad del sistema.

En Spring Boot, la estructura más utilizada es:

- Controller -> Service -> Repository -> Base de datos

-----aquí van las capturas-----

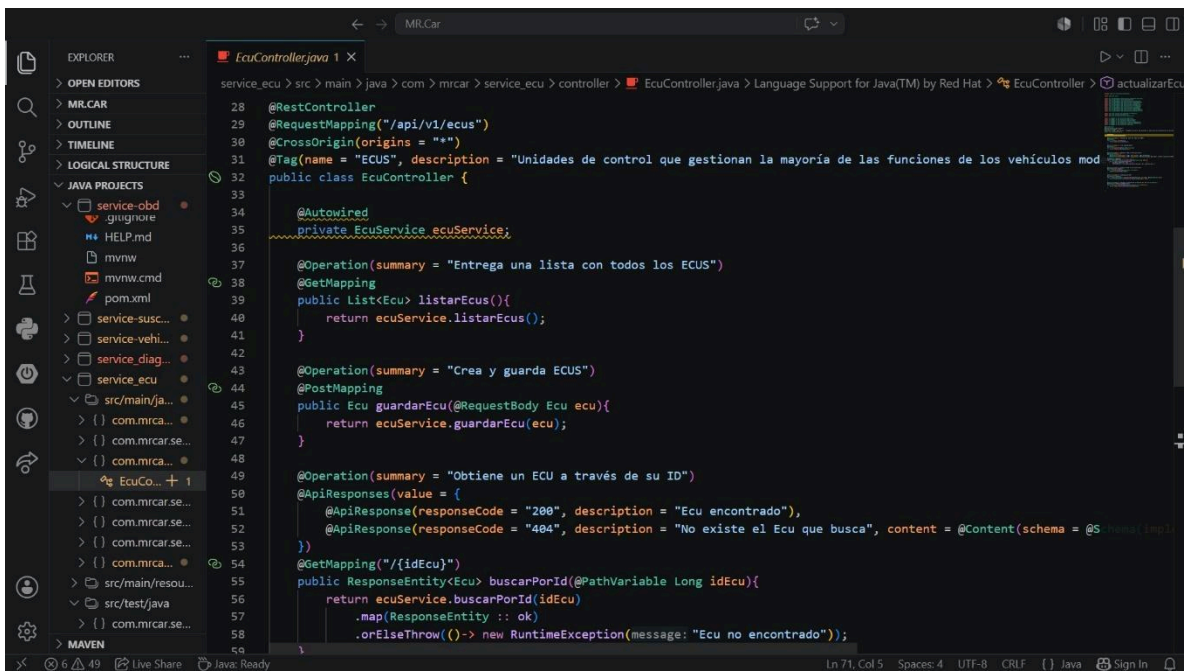
Capa Controller



The screenshot shows the Eclipse IDE with the 'EcuController.java' file open. The Explorer on the left shows a project structure with 'service_ecu' as the active project. The code in the editor includes the following imports and annotations:

```
1 package com.mrcar.service_ecu.controller;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.CrossOrigin;
8 import org.springframework.web.bind.annotation.DeleteMapping;
9 import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RestController;
16
17 import com.mrcar.service_ecu.exception.ErrorResponse;
18 import com.mrcar.service_ecu.model.Ecu;
19 import com.mrcar.service_ecu.service.EcuService;
20
21 import io.swagger.v3.oas.annotations.Operation;
22 import io.swagger.v3.oas.annotations.media.Content;
23 import io.swagger.v3.oas.annotations.media.Schema;
24 import io.swagger.v3.oas.annotations.responses.ApiResponse;
25 import io.swagger.v3.oas.annotations.responses.ApiResponses;
26 import io.swagger.v3.oas.annotations.tags.Tag;
27
28 @RestController
29 @RequestMapping("/api/v1/ecus")
30 @CrossOrigin(origins = "**")
31 @Tag(name = "ECUS", description = "Unidades de control que gestionan la mayoría de las funciones de los vehículos modernos")
32 public class EcuController {
```

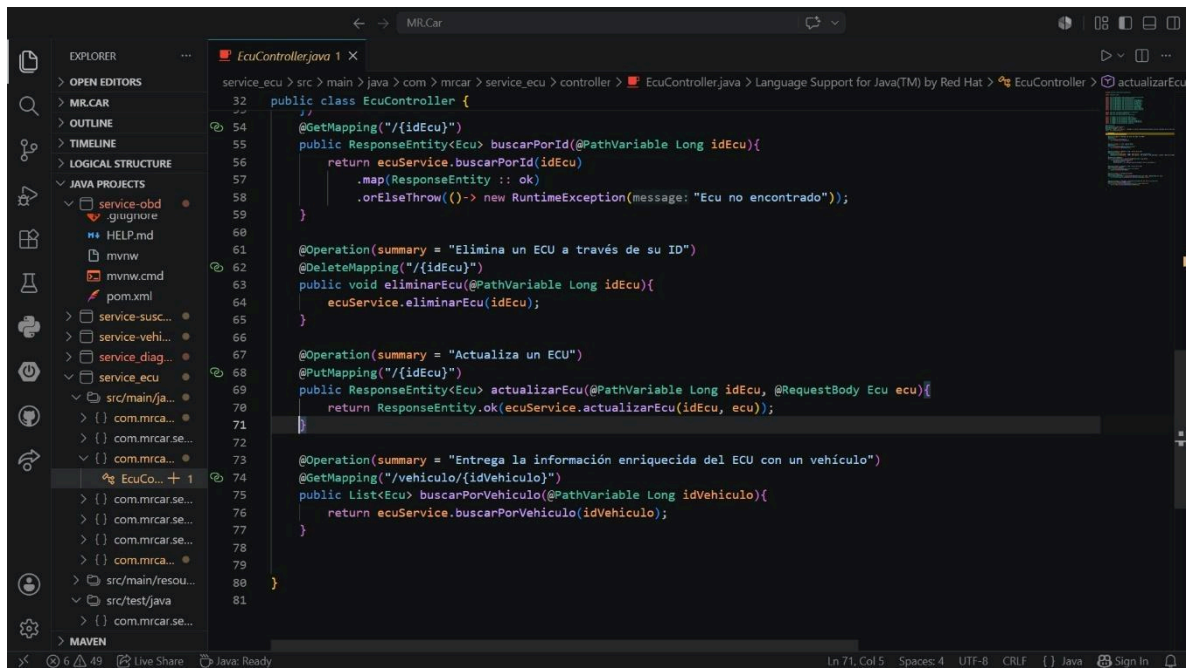
The status bar at the bottom indicates 'Ln 71, Col 5', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'.



The screenshot shows the same IDE with the 'EcuController.java' file open, displaying the implementation of the controller methods. The code includes the following methods:

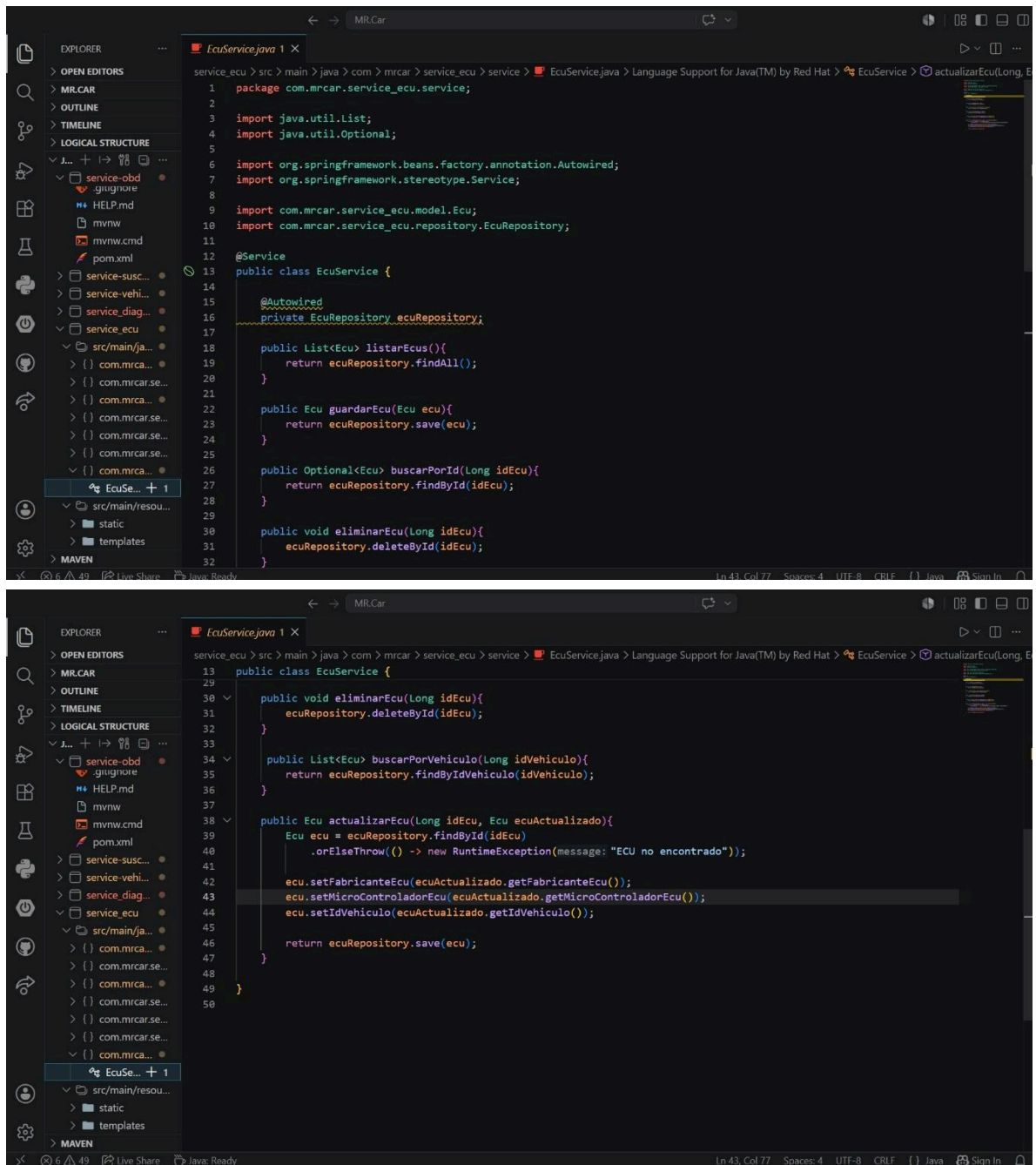
```
33
34 @Autowired
35 private EcuService ecuService;
36
37 @Operation(summary = "Entrega una lista con todos los ECUS")
38 @GetMapping
39 public List<Ecu> listarEcus(){
40     return ecuService.listarEcus();
41 }
42
43 @Operation(summary = "Crea y guarda ECUS")
44 @PostMapping
45 public Ecu guardarEcu(@RequestBody Ecu ecu){
46     return ecuService.guardarEcu(ecu);
47 }
48
49 @Operation(summary = "Obtiene un ECU a través de su ID")
50 @ApiResponses(value = {
51     @ApiResponse(responseCode = "200", description = "Ecu encontrado"),
52     @ApiResponse(responseCode = "404", description = "No existe el Ecu que busca", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
53 })
54 @GetMapping("/{idEcu}")
55 public ResponseEntity<Ecu> buscarPorId(@PathVariable Long idEcu){
56     return ecuService.buscarPorId(idEcu)
57         .map(ResponseEntity::ok)
58         .orElseThrow(() -> new RuntimeException(message: "Ecu no encontrado"));
59 }
```

The status bar at the bottom indicates 'Ln 71, Col 5', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'.

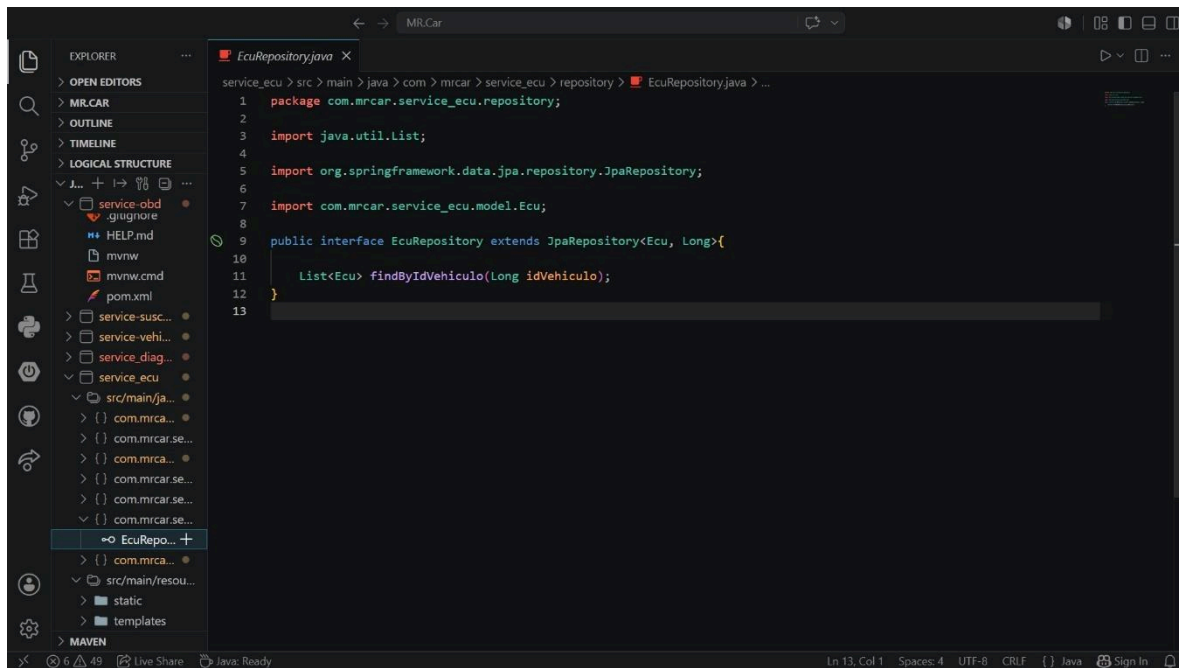


```
32 public class EcuController {
33     //
34     @GetMapping("/{idEcu}")
35     public ResponseEntity<Ecu> buscarPorId(@PathVariable Long idEcu){
36         return ecuService.buscarPorId(idEcu)
37             .map(ResponseEntity::ok)
38             .orElseThrow(() -> new RuntimeException(message: "Ecu no encontrado"));
39     }
40
41     @Operation(summary = "Elimina un ECU a través de su ID")
42     @DeleteMapping("/{idEcu}")
43     public void eliminarEcu(@PathVariable Long idEcu){
44         ecuService.eliminarEcu(idEcu);
45     }
46
47     @Operation(summary = "Actualiza un ECU")
48     @PutMapping("/{idEcu}")
49     public ResponseEntity<Ecu> actualizarEcu(@PathVariable Long idEcu, @RequestBody Ecu ecu){
50         return ResponseEntity.ok(ecuService.actualizarEcu(idEcu, ecu));
51     }
52
53     @Operation(summary = "Entrega la información enriquecida del ECU con un vehículo")
54     @GetMapping("/vehiculo/{idVehiculo}")
55     public List<Ecu> buscarPorVehiculo(@PathVariable Long idVehiculo){
56         return ecuService.buscarPorVehiculo(idVehiculo);
57     }
58 }
```

Capa Service

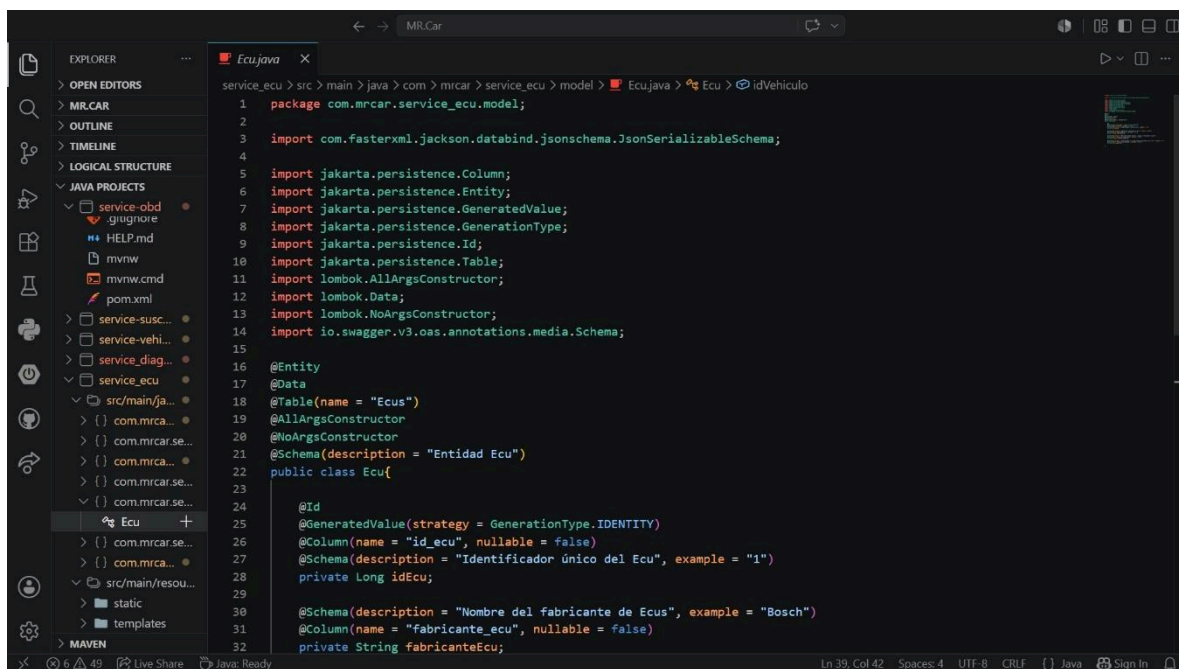


Capa Repository



```
service_ecu > src > main > java > com > mrcar > service_ecu > repository > EcuRepository.java > ...
1 package com.mrcar.service_ecu.repository;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import com.mrcar.service_ecu.model.Ecu;
8
9 public interface EcuRepository extends JpaRepository<Ecu, Long>{
10
11     List<Ecu> findByIdVehiculo(Long idVehiculo);
12 }
13
```

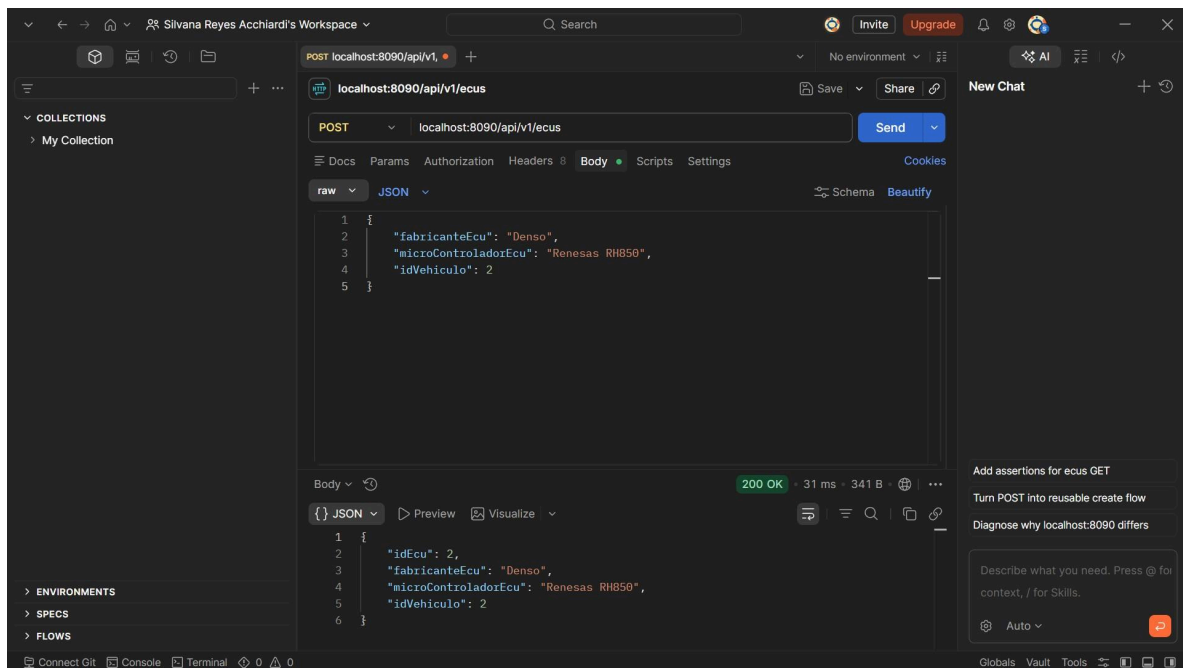
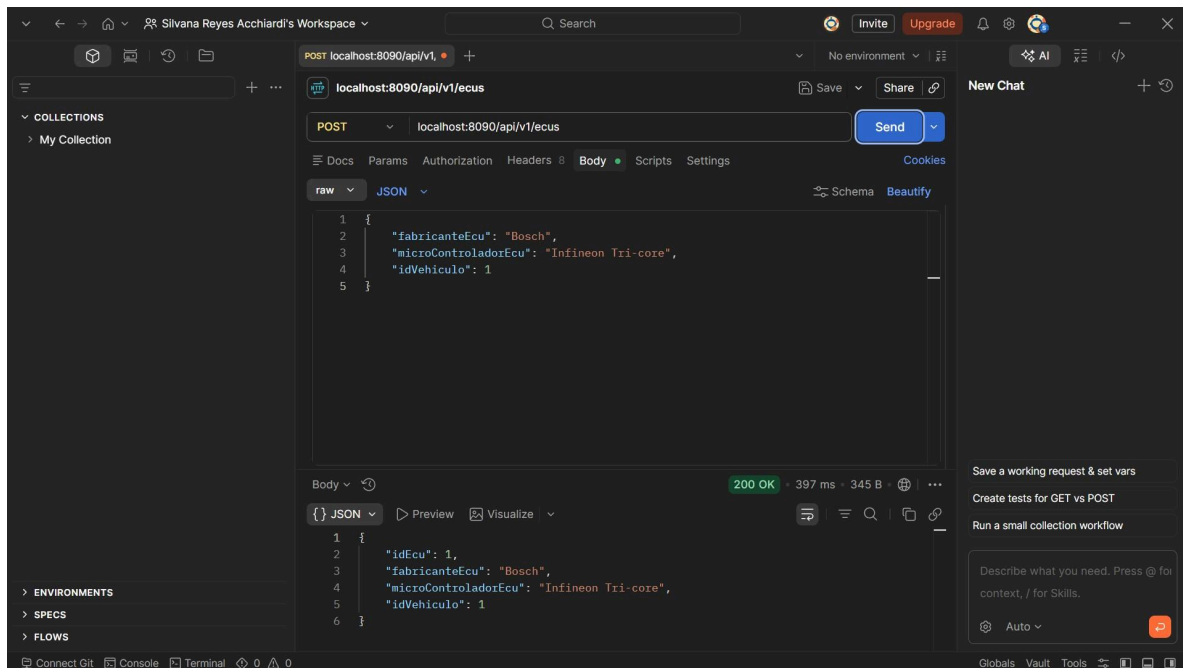
Capa Model



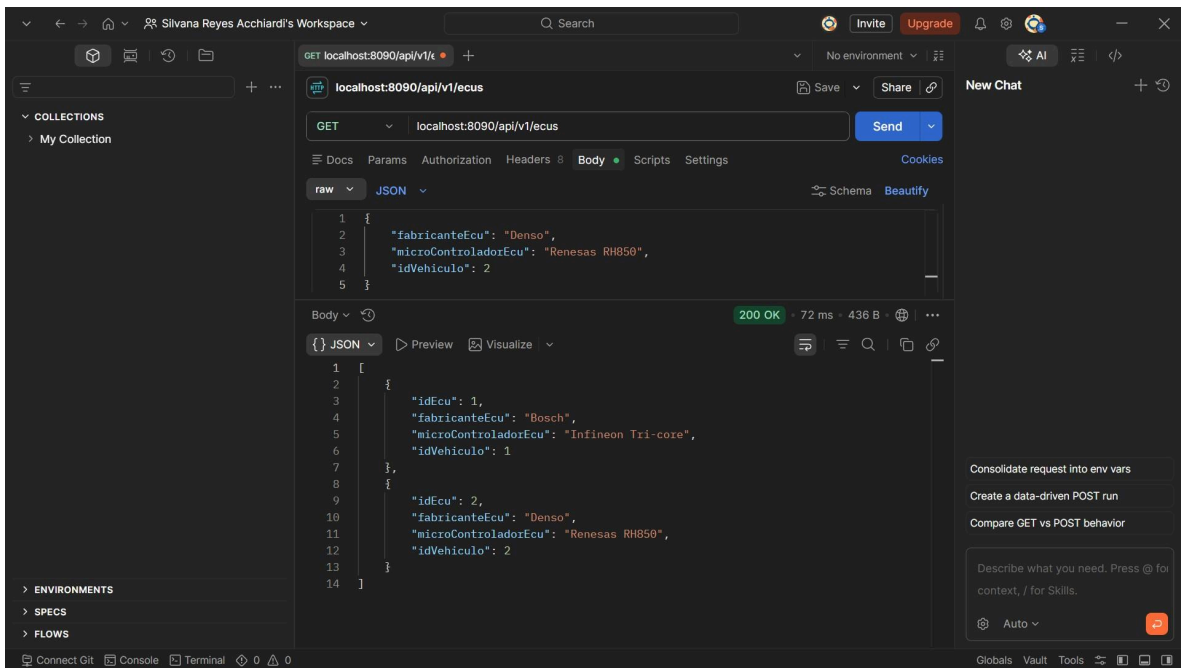
```
service_ecu > src > main > java > com > mrcar > service_ecu > model > Ecu.java > Ecu > idVehiculo
1 package com.mrcar.service_ecu.model;
2
3 import com.fasterxml.jackson.databind.jsonschema.JsonSerializableSchema;
4
5 import jakarta.persistence.Column;
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.GeneratedValue;
8 import jakarta.persistence.GenerationType;
9 import jakarta.persistence.Id;
10 import jakarta.persistence.Table;
11 import lombok.AllArgsConstructor;
12 import lombok.Data;
13 import lombok.NoArgsConstructor;
14 import io.swagger.v3.oas.annotations.media.Schema;
15
16 @Entity
17 @Data
18 @Table(name = "Ecus")
19 @AllArgsConstructor
20 @NoArgsConstructor
21 @Schema(description = "Entidad Ecu")
22 public class Ecu{
23
24     @Id
25     @GeneratedValue(strategy = GenerationType.IDENTITY)
26     @Column(name = "id_ecu", nullable = false)
27     @Schema(description = "Identificador único del Ecu", example = "1")
28     private Long idEcu;
29
30     @Schema(description = "Nombre del fabricante de Ecus", example = "Bosch")
31     @Column(name = "fabricante_ecu", nullable = false)
32     private String fabricanteEcu;
33 }
```

Métodos HTTP:

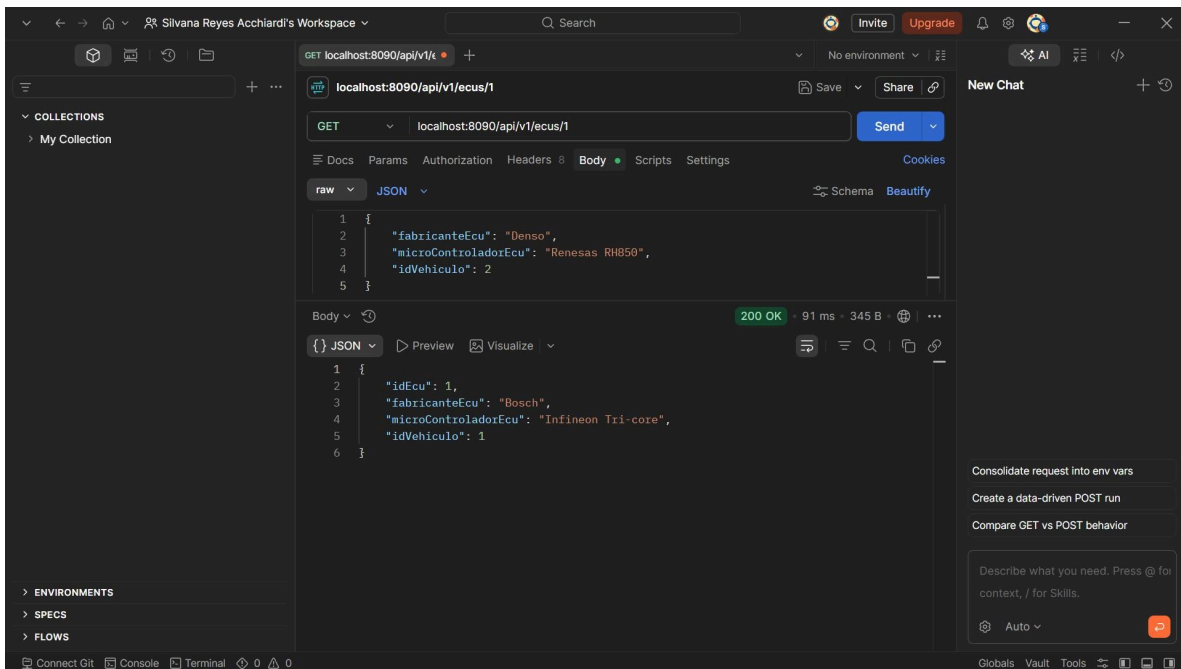
Post:



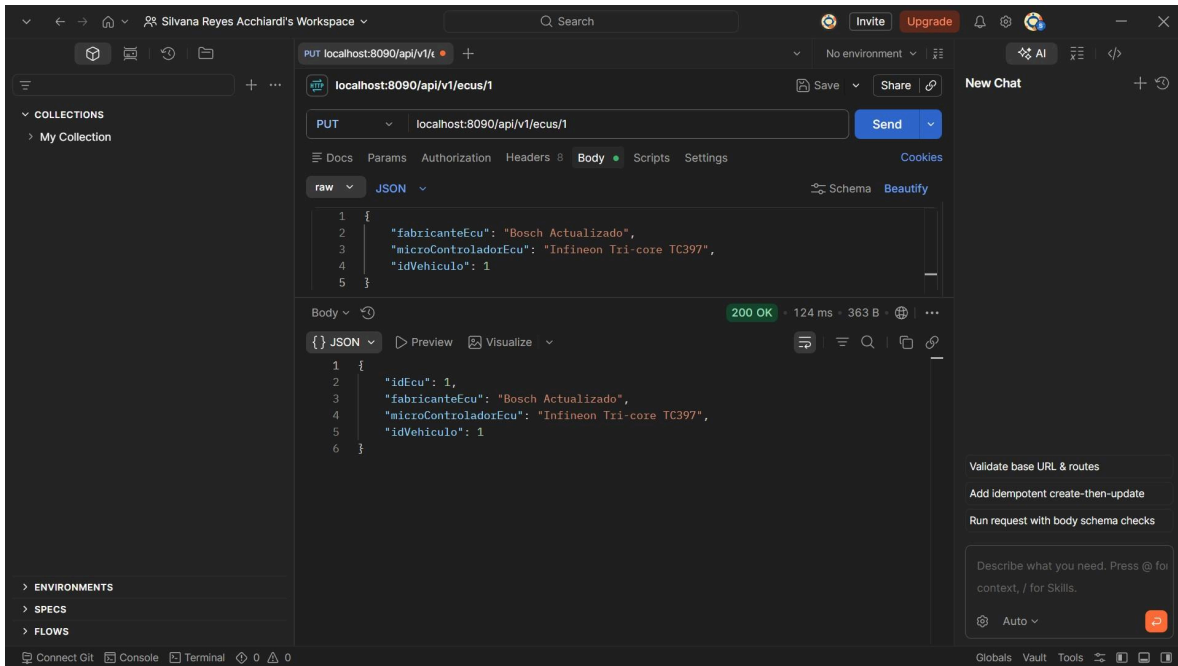
Get:



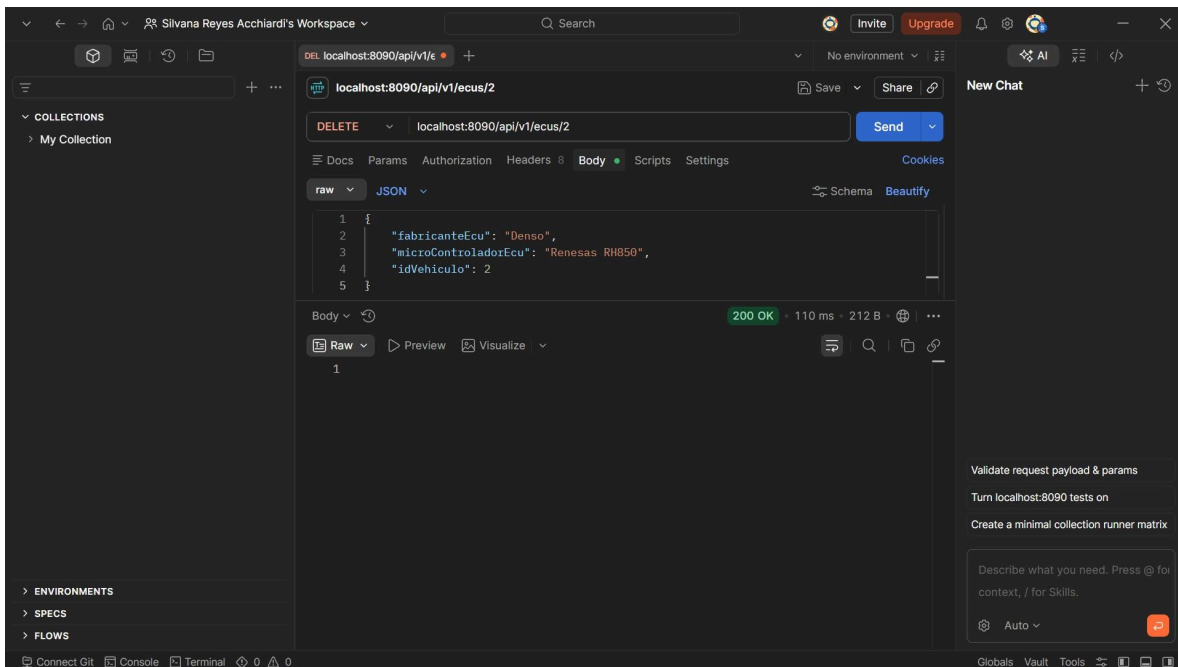
Getid:



Put:



Delete:



11. Comunicación entre Microservicios

La comunicación entre microservicios se implementó a través de WebClient, una herramienta proporcionada por Spring WebFlux que permite realizar peticiones HTTP de un microservicio a otro.

Por ejemplo, el microservicio Historial consulta información de otros servicios:

- Vehículos
- Diagnósticos
- Reparaciones
- OBD

En este caso, Historial solicita información del vehículo utilizando su identificador y la incorpora al historial completo del automóvil.

12. Swagger

Swagger es un conjunto de herramientas que permite documentar y probar servicios REST de manera automática.

A partir de las anotaciones presentes en el código fuente, genera una interfaz web donde se muestran:

- Endpoints disponibles
- Parámetros
- Respuestas
- Códigos HTTP
- Modelos de datos

Esto facilita el desarrollo como las pruebas de los microservicios.

12.1 Implementación Swagger

En Spring Boot se implementa mediante la dependencia:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>
```

Luego se configura en application.properties:

```
#Swagger
springdoc.api-docs.path=/api/v1/diagnosticos/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

Finalmente se documentan los controladores utilizando anotaciones como:

- @Tag

- @Operation
- @ApiResponse
- @Schema

12.2 Evidencias

12.2.1 Microservicio

12.2.2 Microservicio 2

12.2.3 Microservicio 3

13. Testing

El testing consiste en verificar que el software funcione correctamente antes de ser entregado.

Permite detectar errores de manera temprana, reducir fallos en producción y asegurar que los cambios realizados no afecten funcionalidades existentes.

En una arquitectura de microservicios, las pruebas son especialmente importantes porque los microservicios interactúan entre sí y un error en uno puede afectar al resto del sistema.

13.1 Pruebas Unitarias (AAA)

Las pruebas unitarias verifican el funcionamiento de un método o componente de forma aislada.

Se basan en el **patrón AAA (Arrange-Act-Assert)**:

- **Arrange** (Preparar):
Se ejecuta el método que se desea probar.
- **Act** (Actuar):
Se ejecuta el método que se desea probar.
- **Assert** (Verificar)
Se comprueba que el resultado obtenido sea el esperado

Conclusión

Esta definición establece el contexto operativo, técnico y funcional del sistema MR.Car, delimitando su alcance, condiciones de operación y dependencias externas. Esto permite reducir ambigüedades y sirve como base para el diseño detallado, desarrollo e implementación del sistema.